



Customize a view type

Subclass an existing view

Assume we need to create a custom version of a generic view. For example, a kanban view with some extra ribbon-like widget on top (to display some specific custom information). In that case, this can be done in a few steps:

1. Extend the kanban controller/renderer/model and register it in the view registry.

custom_kanban_controller.js

```
import { KanbanController } from "@web/views/kanban/kanban_controller";
import { kanbanView } from "@web/views/kanban/kanban_view";
import { registry } from "@web/core/registry";

// the controller usually contains the Layout and the render method
class CustomKanbanController extends KanbanController {
    static template = "my_module.CustomKanbanView";

    // Your logic here, override or insert new methods...
    // if you override setup(), don't forget to call super.setup()
}

export const customKanbanView = {
    ...kanbanView, // contains the default Renderer/Controller
    Controller: CustomKanbanController,
};

// Register it to the views registry
registry.category("views").add("custom_kanban", customKanbanView);
```

In our custom kanban, we defined a new template. We can either inherit the kanban controller template and add our template pieces or we can define a completely new template.

custom_kanban_controller.xml

```
<xpath expr="//Layout" position="before">
    <div>
        Hello world !
    </div>
</xpath>
</t>
</templates>
```

2. Use the view with the `js_class` attribute in arch.

```
<kanban js_class="custom_kanban">
    <templates>
        <t t-name="card">
            <!--Your comment-->
        </t>
    </templates>
</kanban>
```

The possibilities for extending views are endless. While we have only extended the controller here, you can also extend the renderer to add new buttons, modify how records are presented, or customize the dropdown, as well as extend other components such as the model and `buttonTemplate`.

Create a new view from scratch

Creating a new view is an advanced topic. This guide highlight only the essential steps.

1. Create the controller.

The primary role of a controller is to facilitate the coordination between various components of a view, such as the Renderer, Model, and Layout.

beautiful_controller.js

```
import { Layout } from "@web/search/layout";
import { useService } from "@web/core/utils/hooks";
import { Component, onWillStart, useState } from "@odoo/owl"
```

```

    setup() {
        this.orm = useService("orm");

        // The controller create the model and make it reactive
        // accessed and edited then it'll cause a rerendering
        this.model = useState(
            new this.props.Model(
                this.orm,
                this.props.resModel,
                this.props.fields,
                this.props.archInfo,
                this.props.domain
            )
        );

        onWillStart(async () => {
            await this.model.load();
        });
    }
}

```

The template of the Controller displays the control panel with Layout and also the renderer.

beautiful_controller.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<templates xml:space="preserve">
    <t t-name="my_module.View">
        <Layout display="props.display" className="'h-100 overflow-hidden'">
            <t t-component="props.Renderer" records="model.records"/>
        </Layout>
    </t>
</templates>

```

2. Create the renderer.

The primary function of a renderer is to generate a visual representation of data by rendering the view that includes records.

```
static template = "my_module.Renderer";
}
```

beautiful_renderer.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<templates xml:space="preserve">
  <t t-name="my_module.Renderer">
    <t t-esc="props.propsYouWant"/>
    <t t-foreach="props.records" t-as="record" t-key="r">
      // Show records
    </t>
  </t>
</templates>
```

3. Create the model.

The role of the model is to retrieve and manage all the necessary data in the view.

beautiful_model.js

```
import { KeepLast } from "@web/core/utils/concurrency";

export class BeautifulModel {
  constructor(orm, resModel, fields, archInfo, domain) {
    this.orm = orm;
    this.resModel = resModel;
    // We can access arch information parsed by the beautiful_renderer
    const { fieldFromTheArch } = archInfo;
    this.fieldFromTheArch = fieldFromTheArch;
    this.fields = fields;
    this.domain = domain;
    this.keepLast = new KeepLast();
  }

  async load() {
    // The keeplast protect against concurrency call
    const { length, records } = await this.keepLast.add(
      this.orm.webSearchRead(this.resModel, this.domain)
    );
  }
}
```

Note

For advanced cases, instead of creating a model from scratch, it is also possible to use `RelationalModel`, which is used by other views.

4. Create the arch parser.

The role of the arch parser is to parse the arch view so the view has access to the information.

beautiful_arch_parser.js

```
import { XMLParser } from "@web/core/utils/xml";

export class BeautifulArchParser extends XMLParser {
  parse(arch) {
    const xmlDoc = this.parseXML(arch);
    const fieldFromTheArch = xmlDoc.getAttribute("field");
    return {
      fieldFromTheArch,
    };
  }
}
```

5. Create the view and combine all the pieces together, then register the view in the views registry.

beautiful_view.js

```
import { registry } from "@web/core/registry";
import { BeautifulController } from "../beautiful_controller";
import { BeautifulArchParser } from "../beautiful_arch_parser";
import { BeautifulModel } from "../beautiful_model";
import { BeautifulRenderer } from "../beautiful_renderer";

export const beautifulView = {
  type: "beautiful",
```

```

    ArchParser: BeautifulArchParser,
    Model: BeautifulModel,
    Renderer: BeautifulRenderer,

    props(genericProps, view) {
        const { ArchParser } = view;
        const { arch } = genericProps;
        const archInfo = new ArchParser().parse(arch);

        return {
            ...genericProps,
            Model: view.Model,
            Renderer: view.Renderer,
            archInfo,
        };
    },
};

registry.category("views").add("beautifulView", beautifulView);

```

6. Declare the **view** in the arch.

```

...
<record id="my_beautiful_view" model="ir.ui.view">
    <field name="name">my_view</field>
    <field name="model">my_model</field>
    <field name="arch" type="xml">
        <beautiful fieldFromTheArch="res.partner"/>
    </field>
</record>
...

```

